

Module 02: Exploratory Data Analysis (EDA)

Lecturer: Abdan Hafidz

Topic: Data Visualization & Statistical Inference for ML

1 Introduction: The Detective Work of Machine Learning

Exploratory Data Analysis (EDA) is the critical first step in any machine learning project. It is the process of "listening" to the data before attempting to model it. While many beginners are tempted to rush directly into training complex algorithms like Neural Networks, seasoned practitioners understand that the quality of the output is strictly bound by the quality of the input—a principle known as **Garbage In, Garbage Out (GIGO)**.

EDA is not merely about generating aesthetically pleasing charts. It is a rigorous investigative process designed to uncover the underlying structure of the dataset, identify anomalies, test initial hypotheses, and check assumptions. By visualizing distributions and relationships, we transform raw numbers into actionable insights, allowing us to choose the correct model and preprocessing strategies.

2 Statistical Foundations: Measuring Centrality and Spread

To interpret visualizations effectively, we must first ground ourselves in the mathematical language of descriptive statistics. These metrics provide a summary of the data's "location" and "shape."

2.1 Central Tendency: The Quest for the "Typical" Value

When we want to describe a "typical" data point, we look for the center of the distribution. The two most common metrics are the **Mean** (μ) and the **Median**.

The **Mean** is the arithmetic average, calculated by summing all values and dividing by the count (N). While mathematically convenient, the mean has a fatal flaw: it is highly sensitive to outliers.

Consider a small village with 5 residents earning reasonable salaries: \$30k, \$35k, \$40k, \$45k, and \$50k. The mean income is \$40k, which accurately reflects the village. Now, imagine a billionaire like Elon Musk moves in, earning \$1,000,000k. The new mean skyrockets to over \$160k. Does this mean the average villager is now rich? No. The mean has been corrupted by a single outlier.

This is where the **Median** shines. The median is the middle value when the data is sorted. In our village example, even with the billionaire, the median income remains near \$45k. This property is called **Robustness**. In EDA, if the Mean and Median are significantly different, it is a strong signal that the data is skewed and contains outliers.

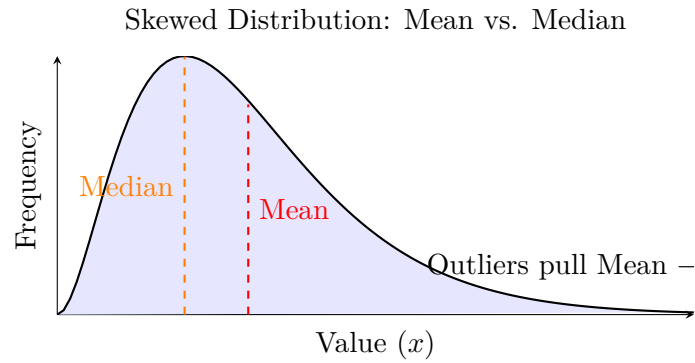


Figure 1: In a Right-Skewed distribution (like income), the Mean (Red) is pulled towards the outliers, while the Median (Orange) remains robust.

2.2 Dispersion: Quantifying Uncertainty

Knowing the center is not enough; we must also understand how spread out the data is. This spread represents variability or uncertainty.

The **Variance** (σ^2) measures the average squared deviation of each data point from the mean.

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

We square the differences to ensure that negative and positive deviations do not cancel each other out. However, squaring the values changes the unit (e.g., from "dollars" to "dollars squared"), which is hard to interpret.

Therefore, we take the square root of the variance to get the **Standard Deviation** (σ). This brings the metric back to the original unit, allowing us to say, for example, "The average house price is \$500k, plus or minus \$50k."

3 Univariate Analysis: Understanding Distributions

Univariate analysis involves examining each feature in isolation to understand its probability distribution. This helps us answer questions like: "Is the data normal?" or "Are there extreme values?"

3.1 The Histogram: Visualizing Probability Density

A histogram groups continuous data into "bins" and plots the frequency of observations in each bin. It is an empirical approximation of the Probability Density Function (PDF).

When analyzing a histogram, we look for **Skewness**.

- A **Symmetric (Bell-Shaped)** distribution implies normality, which is an assumption for many algorithms like Linear Regression.
- A **Right-Skewed** distribution (long tail to the right) is common in financial data (e.g., House Prices, Salaries), where a few high values pull the mean upwards.
- A **Left-Skewed** distribution (long tail to the left) is less common but appears in data with a hard upper limit (e.g., Age at death).

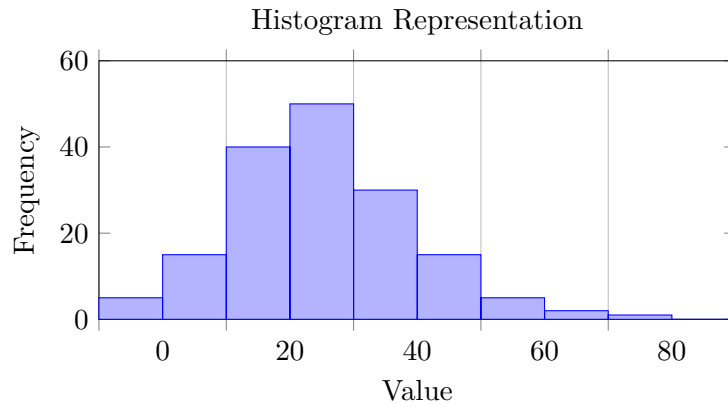


Figure 2: A Histogram shows the frequency distribution. This example is slightly right-skewed.

Python Implementation:

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Visualizing distribution of House Prices
5 plt.figure(figsize=(8, 5))
6 sns.histplot(df['median_house_value'], kde=True, color='blue')
7 plt.title('Distribution of House Values (Right-Skewed)')
8 plt.xlabel('Value ($)')
9 plt.show()

```

3.2 The Box Plot: Anatomy of an Outlier

While histograms show the shape, **Box Plots** are designed to highlight outliers and robustness. A box plot is built on the concept of **Quartiles**, which divide the sorted data into four equal parts.

The Anatomy of a Box Plot



Figure 3: The Box represents the Interquartile Range (IQR), containing the middle 50% of data. Whiskers extend to $1.5 \times IQR$. Points beyond whiskers are considered statistical outliers.

The "Box" represents the **Interquartile Range (IQR)**, defined as $Q3 - Q1$. The whiskers extend to $1.5 \times IQR$ from the box edges. Any data point lying outside these whiskers is statistically flagged as an outlier. This method is superior to standard deviation for outlier detection because it relies on the median and quartiles, which are not influenced by the outliers themselves.

3.3 The Count Plot: Categorical Frequency

For discrete, categorical features (like `ocean_proximity` or `soil_type`), histograms are inappropriate. Instead, we use **Count Plots** (Bar Charts). These visualize the frequency of each unique category.

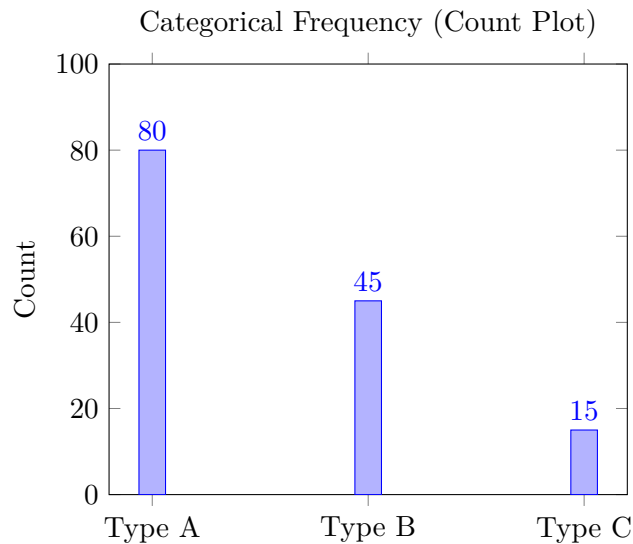


Figure 4: A Count Plot displaying the frequency of three categories. Notice the class imbalance: Type A is much more frequent than Type C.

Insight to Seek:

- **Class Imbalance:** Is one category dominating the dataset? For example, if 90% of the data is "Inland" and only 10% is "Near Ocean", a model might simply guess "Inland" every time and still achieve 90% accuracy. This is a trap known as the **Accuracy Paradox**.
- **Cardinality:** How many unique categories are there? High cardinality (e.g., 1,000 unique cities) might require specialized encoding techniques like Target Encoding instead of One-Hot Encoding.

Python Implementation:

```
1 plt.figure(figsize=(8, 5))
2 sns.countplot(x='ocean_proximity', data=df, palette='viridis')
3 plt.title('Frequency of Locations')
4 plt.show()
```

4 Bivariate Analysis: Hunting for Relationships

Machine learning models learn by finding patterns between features (x) and targets (y). Bivariate analysis allows us to visualize these potential relationships.

4.1 Scatter Plots and Heteroscedasticity

A **Scatter Plot** places one variable on the x-axis and another on the y-axis. It is the most direct way to visualize the relationship $y = f(x)$.

When analyzing a scatter plot, we look for: 1. **Linearity:** Do the points follow a straight line? If yes, Linear Regression is a good candidate. If they follow a curve (e.g., quadratic), we might need non-linear models or feature engineering (x^2). 2. **Heteroscedasticity:** This is a fancy term for "uneven spread." Imagine plotting "Income" vs. "Spending." For low income, spending is tightly clustered (low variance). For high income, spending varies wildly (some save, some spend all). This creates a "fan" or "cone" shape in the plot. Models trained on such data will have unstable error terms.

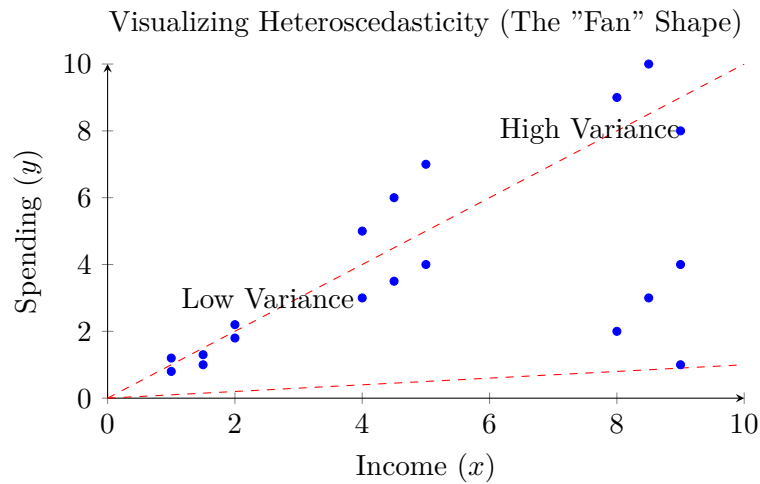


Figure 5: Heteroscedasticity: As Income (x) increases, the variance of Spending (y) increases, creating a "fan" shape. This violates assumptions of simple linear models.

4.2 Correlation Heatmaps

To summarize relationships between many continuous variables, we calculate the **Correlation Matrix**. Each cell (i, j) contains the Pearson correlation coefficient r between feature i and feature j .

$$r_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$$

Visualizing this matrix as a **Heatmap** allows us to spot two things instantly: 1. **Predictive Signals**: Features with high correlation (absolute value close to 1) with the Target variable. These are the most valuable features. 2. **Multicollinearity**: Features that are highly correlated with *each other* (e.g., "Garage Area" and "Garage Cars"). This is redundant information and can destabilize linear models. We often drop one of the two.

	x_1	x_2	x_3
x_1	1.0	0.8	-0.2
x_2	0.8	1.0	-0.1
x_3	-0.2	-0.1	1.0

Figure 6: Conceptual Heatmap. Dark Red indicates strong positive correlation. Here, x_1 and x_2 are highly correlated (0.8), suggesting Multicollinearity.

```

1 # Generating a Correlation Heatmap
2 plt.figure(figsize=(10, 8))
3 corr_matrix = df.corr()
4 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
5 plt.title('Correlation Heatmap: Red = Positive, Blue = Negative')
6 plt.show()

```

5 Data Quality Case Study: Missing Values

Real-world data is never clean. A common issue is **Missing Values (NaNs)**. How we handle them depends entirely on *why* they are missing.

5.1 MCAR (Missing Completely At Random)

Imagine a temperature sensor that runs out of battery randomly. The missing data has no pattern related to the temperature itself. In this case, it is safe to either drop the rows (if few) or impute with the mean.

5.2 MNAR (Missing Not At Random)

Now consider a survey about income. Wealthy people might be more likely to leave the "Income" field blank due to privacy concerns. Here, the *missingness itself* carries information (Missing = High Income). If we simply fill these NaNs with the average income of the population, we will severely underestimate the true wealth, introducing **Bias**.

Imputation Strategy:

- If the distribution is Normal: Fill with **Mean**.
- If the distribution is Skewed (contains outliers): Fill with **Median**.
- If the data is Categorical: Fill with **Mode** (most frequent).